Algorithm Engineering
Woche 1, Aufgabe 2

Jan Vogt

Kapitel 1 aus Computer Systems: A Programmer's Perspective

Kapitel 1 aus dem Buch „Computer Systems: A Programmer's Perspective“ von Bryant und O'Hallaron gibt einen Überblick darüber, was im Rechner alles passiert, wenn man ein einfaches hello-Programm schreibt, kompiliert und ausführt. Ich gehe hier auf zwei Themen aus dem Kapitel genauer ein, nämlich wie Informationen im Rechner dargestellt werden und warum Caches so wichtig sind.

1. Information ist Bits und Kontext

Am Anfang des Kapitels wird gezeigt, wie die Datei `hello.c` eigentlich gespeichert ist. Sie besteht nur aus Bytes, wobei jedes Byte über den ASCII-Code für ein Zeichen steht. Das erste Byte hat zum Beispiel den Wert 35, was dem Zeichen `#` entspricht, und am Ende jeder Zeile steht ein unsichtbares Newline-Zeichen mit dem Wert 10.

Die wichtige Aussage dahinter ist, dass im Rechner alles nur aus Bits besteht, egal ob es sich um eine Datei, ein Programm im Speicher oder Daten aus dem Netzwerk handelt. Was die Bits bedeuten, hängt nur vom Kontext ab, also davon, wie man sie liest. Dieselben Bytes können eine Ganzzahl, eine Kommazahl, ein Text oder sogar ein Maschinenbefehl sein.

Das sieht man gut an einem kleinen Beispiel. Die vier Buchstaben `hell` sind die Bytes 104, 101, 108 und 108. Liest man diese vier Bytes auf einem normalen x86-Rechner als `int`, kommt die Zahl 1819043176 heraus. Liest man sie dagegen als `float`, ergibt sich ungefähr $1{,}14 \cdot 10^{27}$. Im Speicher ist dabei nichts anders, nur die Interpretation ändert sich.

Außerdem wird im Kapitel erwähnt, dass Zahlen im Rechner nicht dasselbe sind wie Zahlen in der Mathematik, sondern nur endliche Annäherungen, die sich manchmal unerwartet verhalten. Ein bekanntes Beispiel ist, dass `0.1 + 0.2` bei `double` nicht genau `0.3` ergibt, sondern `0.30000000000000004`. Das ist mir auch im Zusammenhang mit der Vorlesung aufgefallen: Beim parallelen π-Programm summiert jeder Thread erst seinen eigenen Teil auf und danach werden die Teilsummen addiert. Weil sich dadurch die Reihenfolge der Additionen ändert, können sich die letzten Nachkommastellen von π je nach Anzahl der Threads leicht unterscheiden, obwohl mathematisch dasselbe berechnet wird.

2. Caches und die Speicherhierarchie

Am hello-Beispiel sieht man, dass ein Rechner sehr viel Zeit damit verbringt, Daten von einem Ort an einen anderen zu kopieren. Das Programm liegt zuerst auf der Festplatte, wird dann in den Hauptspeicher geladen und von dort aus Stück für Stück in den Prozessor kopiert. Der eigentliche Text „hello, world“ geht sogar den Weg von der Festplatte über den Hauptspeicher bis zum Bildschirm. Aus Sicht des Programmierers ist dieses ganze Kopieren eigentlich nur Overhead.

Das Problem ist, dass große Speicher langsam sind und schnelle Speicher teuer. Laut Buch kann eine Festplatte zwar etwa 1000 mal größer sein als der Hauptspeicher, aber das Lesen eines Wortes dauert ungefähr 10.000.000 mal länger. Auf der anderen Seite sind die Register nur ein paar hundert Bytes groß, dafür kann der Prozessor aus ihnen fast 100 mal schneller lesen als aus dem Hauptspeicher. Dazu kommt, dass Prozessoren über die Jahre viel schneller geworden sind als der Hauptspeicher, sodass dieser Abstand immer größer wird.

Als Lösung gibt es Caches. Das sind kleine und schnelle Speicher direkt am Prozessor, in denen Daten liegen, die wahrscheinlich bald wieder gebraucht werden. Der L1-Cache ist fast so schnell wie die Register, der L2-Cache ist größer aber etwas langsamer und viele Prozessoren haben noch einen L3-Cache. Das funktioniert, weil Programme meistens auf Daten zugreifen, die nah beieinander liegen oder gerade erst benutzt wurden (Lokalität).

Das Ganze bildet eine Hierarchie, bei der jede Ebene ein Cache für die Ebene darunter ist. Die Register sind ein Cache für den L1-Cache, der L1-Cache für den L2-Cache und so weiter, bis hin zum Hauptspeicher, der wiederum ein Cache für die Festplatte ist. Je weiter man nach unten geht, desto größer, langsamer und billiger pro Byte wird der Speicher.

Wichtig sind dabei vor allem die großen Unterschiede zwischen den Ebenen. Im Buch steht, dass man ein Programm um eine Größenordnung schneller machen kann, wenn man beim Programmieren auf die Caches achtet. Das heißt, dass die Laufzeit nicht nur davon abhängt, wie viele Rechenschritte ein Algorithmus braucht, sondern auch davon, wo die Daten gerade liegen. Genau das sagt auch die Motivation der Vorlesung, nämlich dass die reine Komplexität eines Algorithmus nicht alles über seine echte Laufzeit aussagt.

Die Zahlen im Buch sind allerdings schon etwas älter. Heute haben die meisten Rechner eine SSD statt einer normalen Festplatte, die deutlich schneller ist, aber im Vergleich zum Hauptspeicher trotzdem sehr langsam bleibt. Auch die Caches sind inzwischen viel größer geworden. Der Prozessor im PC-Pool hat zum Beispiel einen L3-Cache mit 33 MB, den sich alle Kerne teilen.